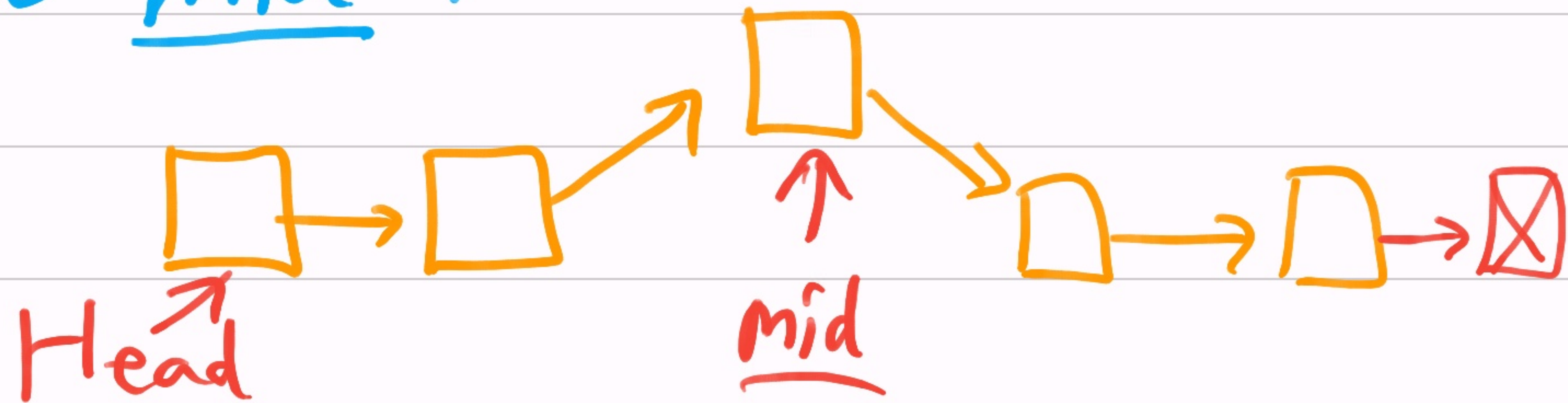
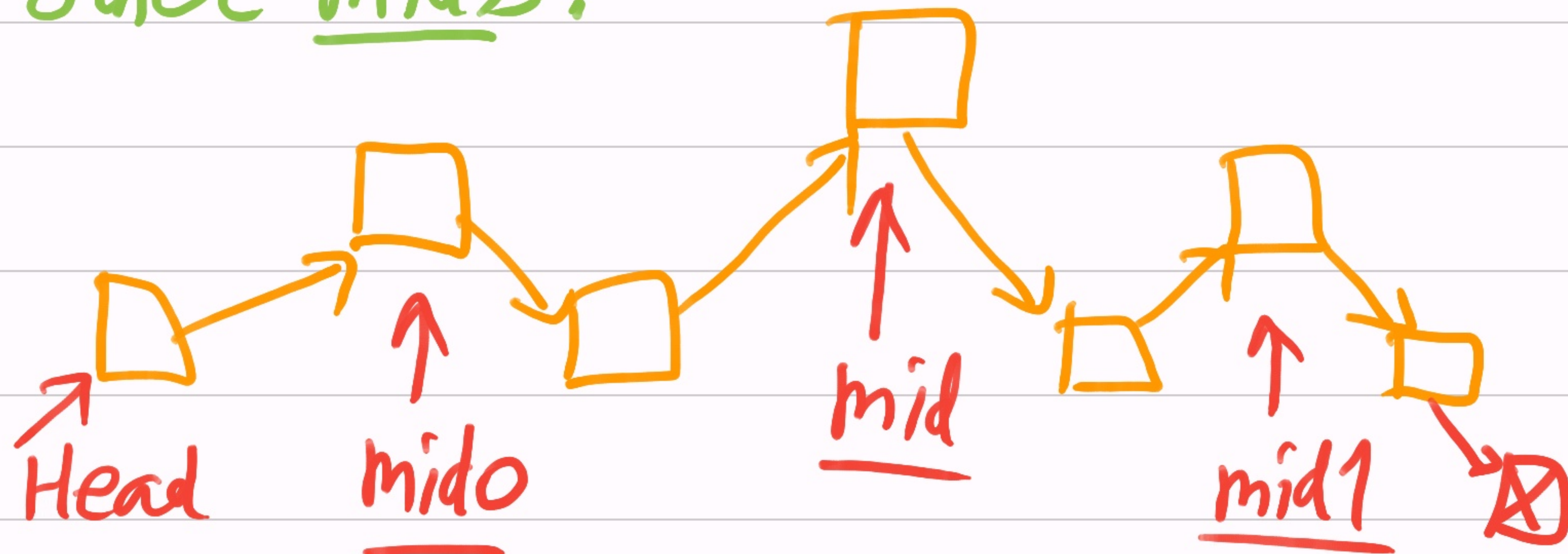
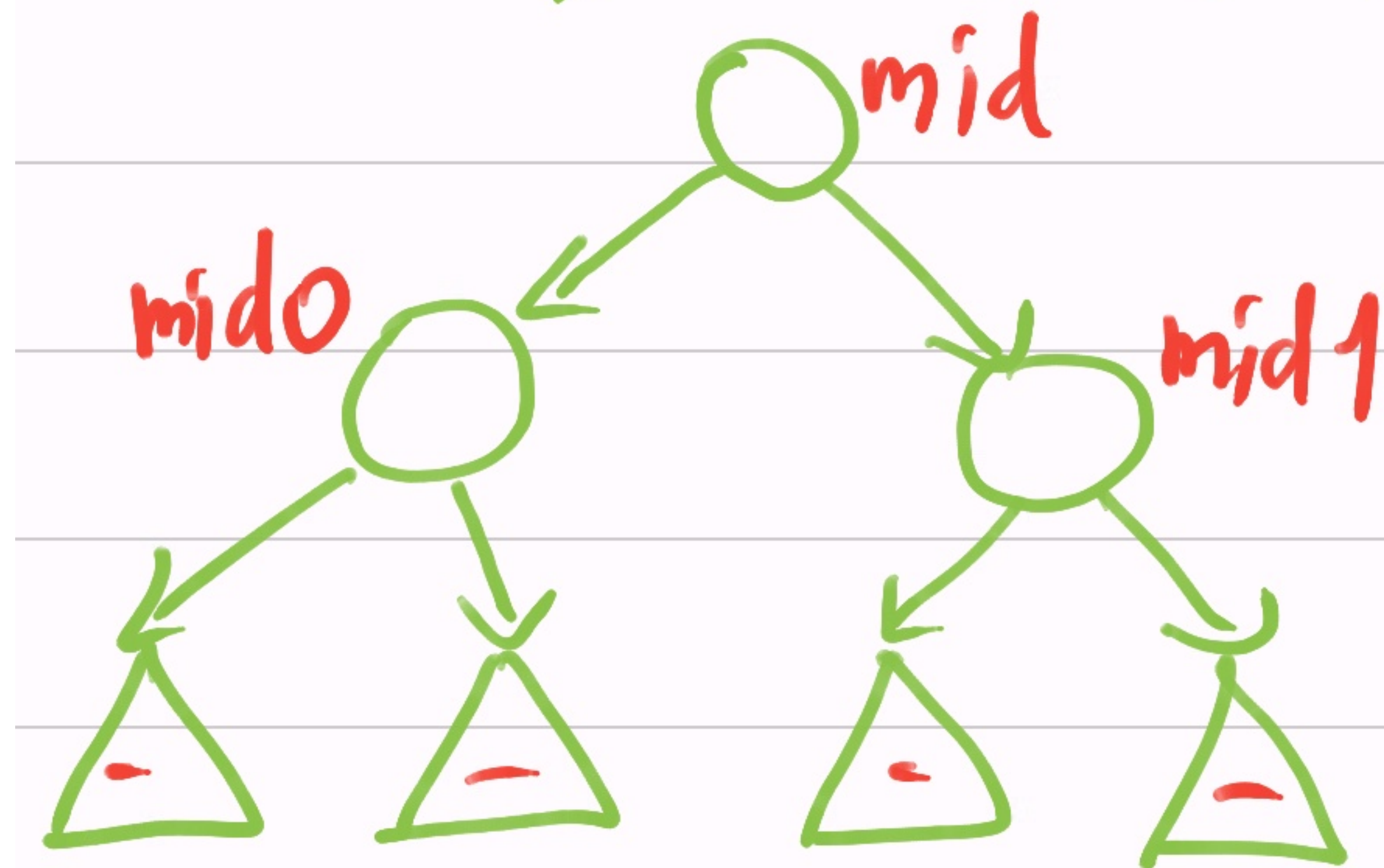


▷ To do (fast) search, in link-list, all we need is ptr. to mid.



- Next, we need the other mids:



▷ Binary search gives us the invention of Binary Tree (a new data structure).

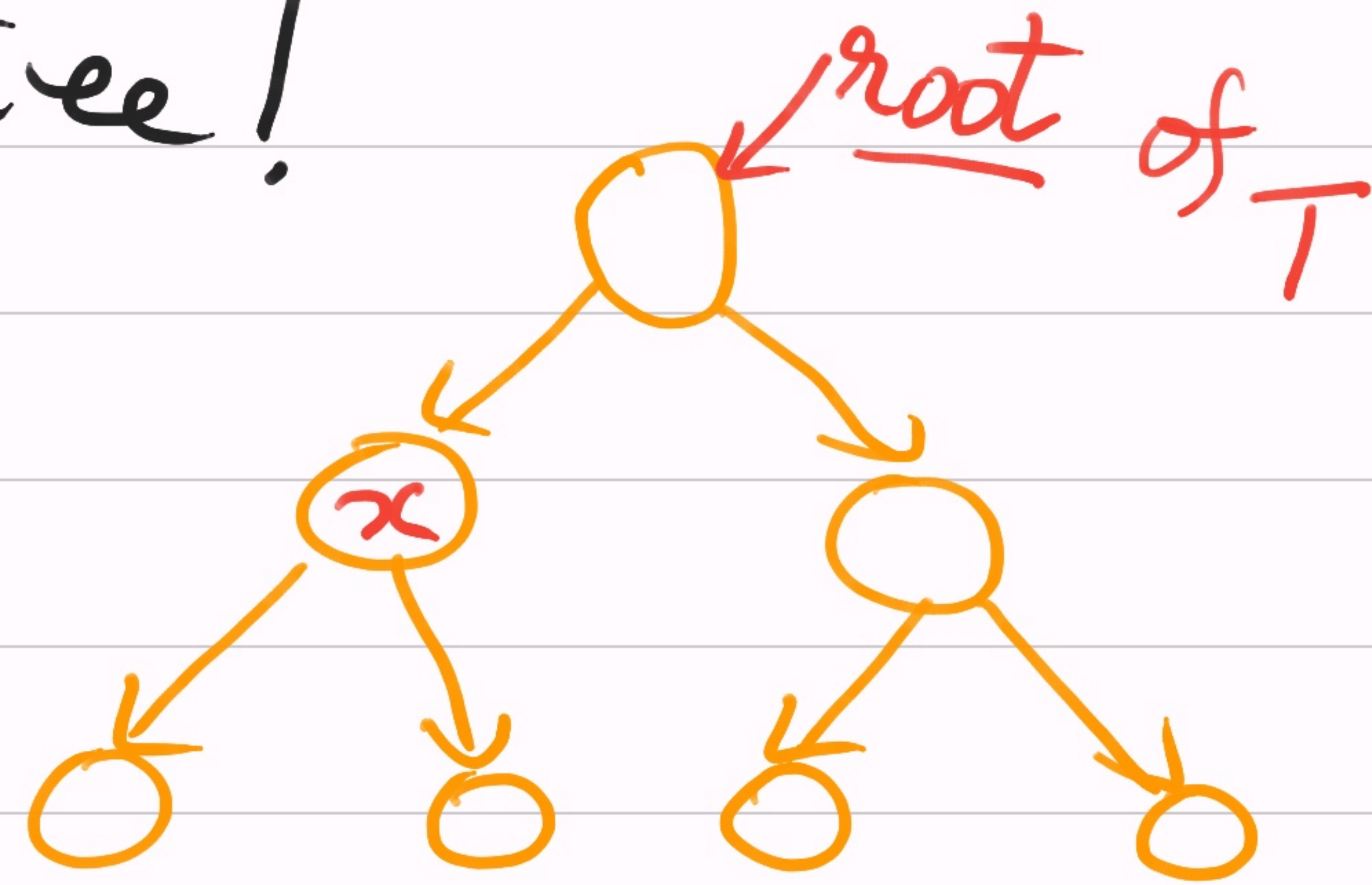
— It's an inverted real tree!

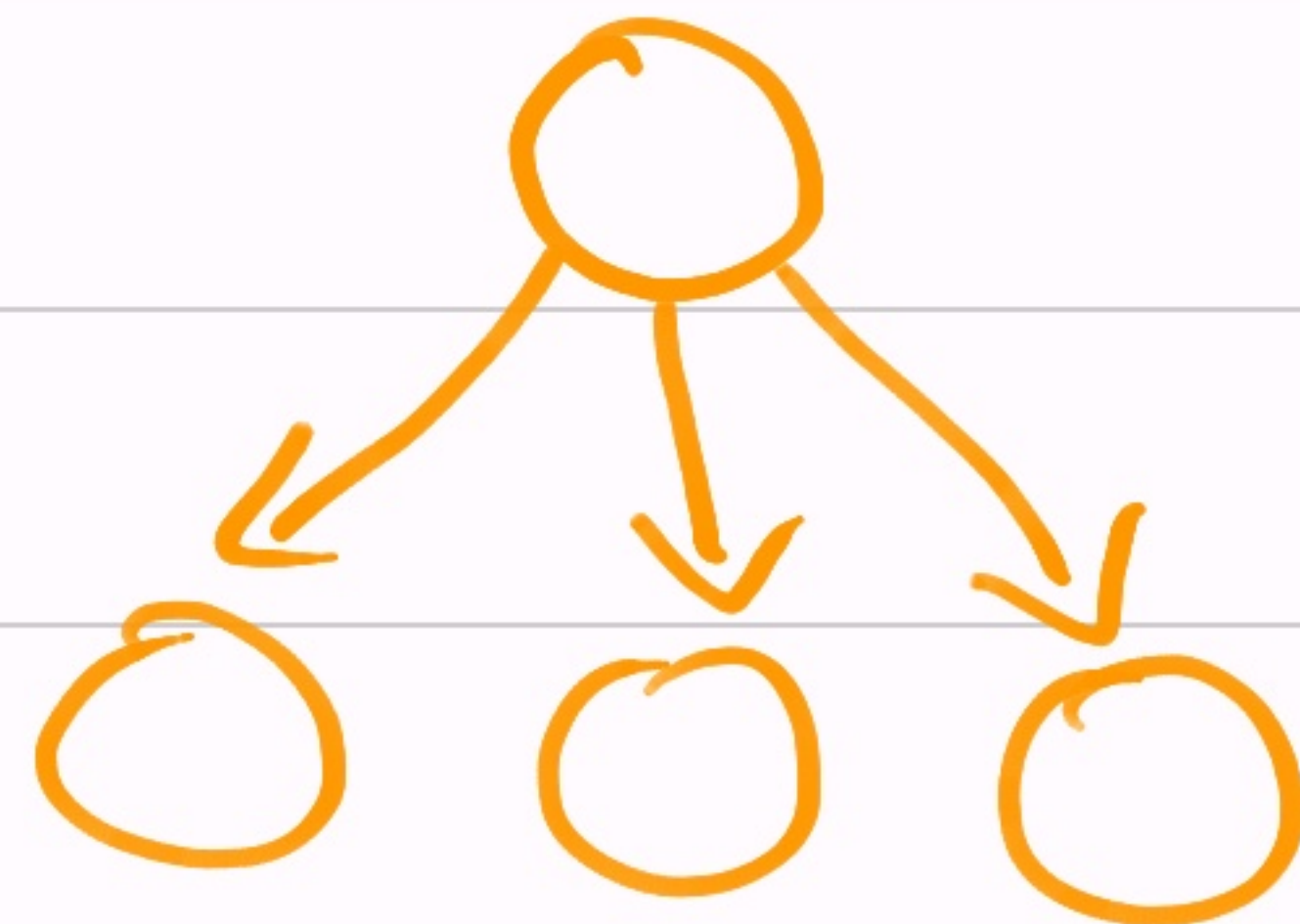
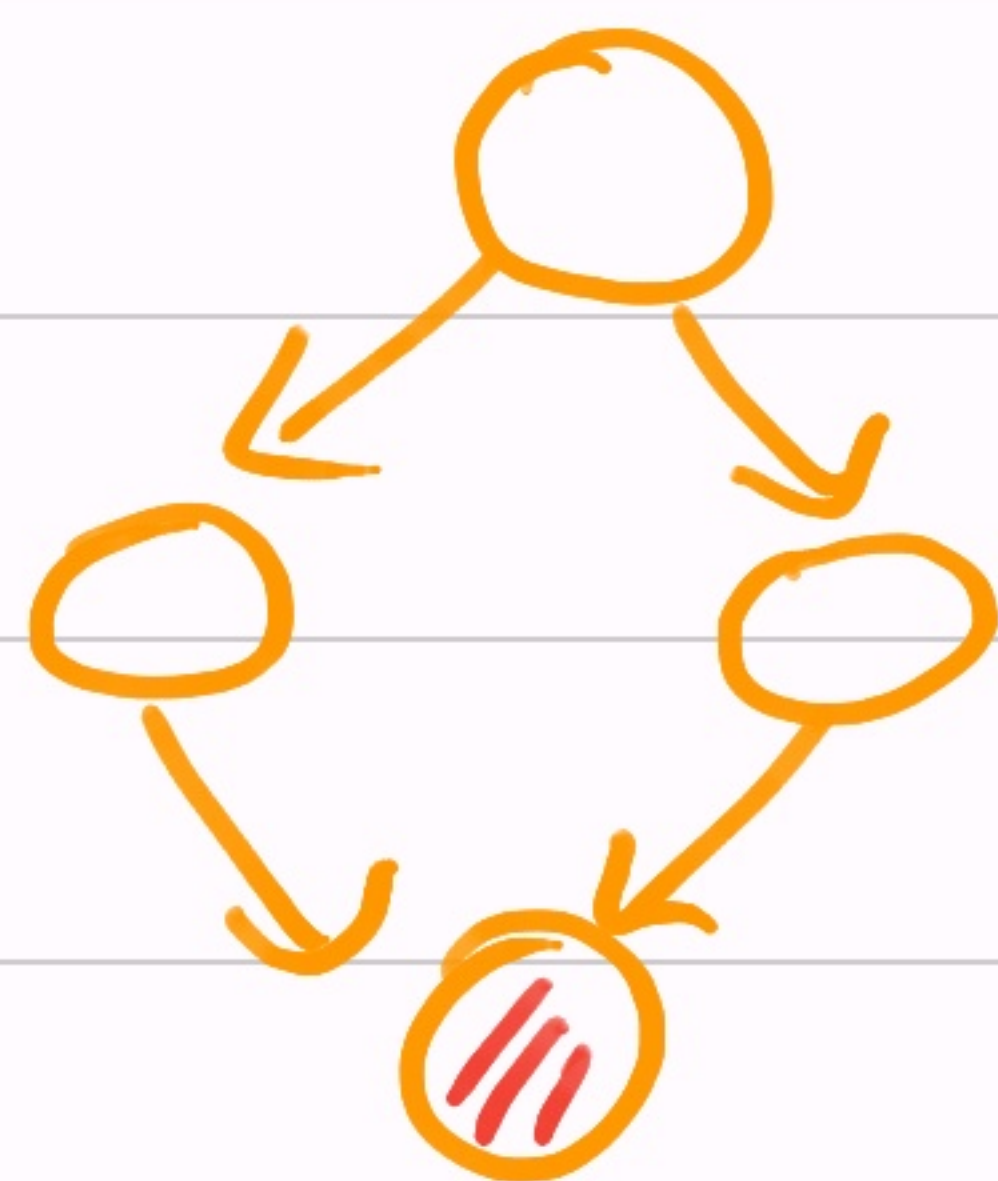
Defn: • Binary tree T has root, edges, nodes, leaves.

• Each node stores: a value, & a ptr. to left & a ptr. to right.

• Any node: # incoming edges = 0, 1.

outgoing edges = 0, 1, 2. [Binary]



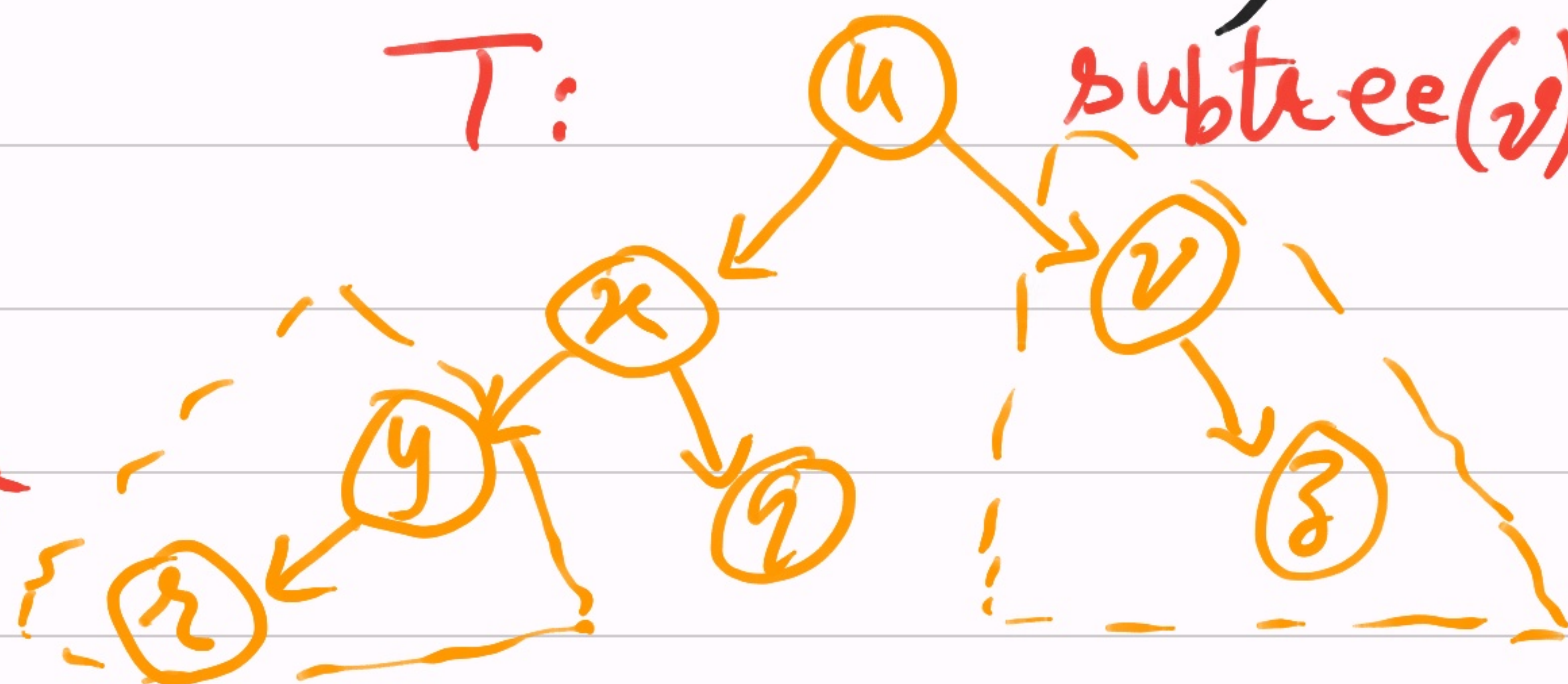


Not a tree ; Tree but non-binary ; Not a tree as it's a loop.

— Some terminology you'll see in this course, all the time:

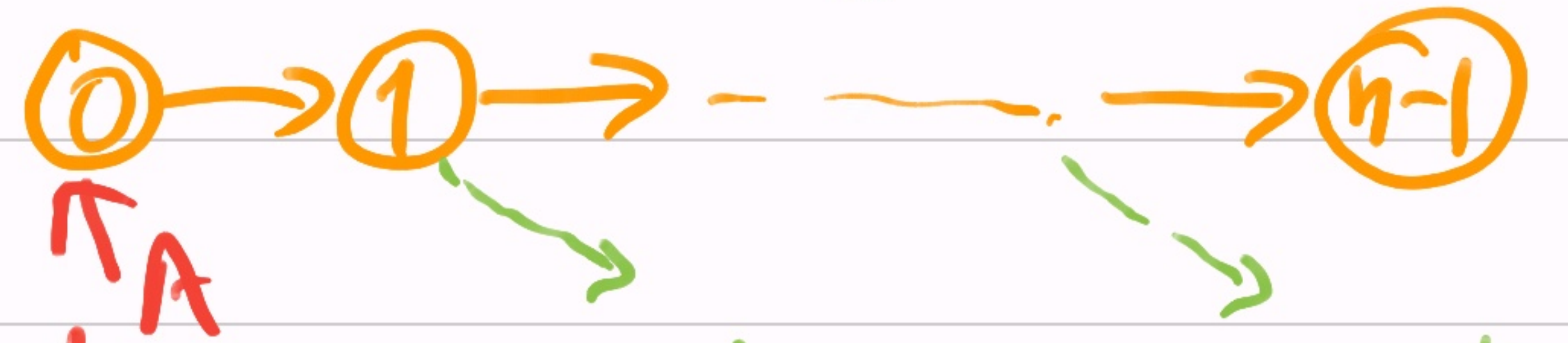
- $\text{parent}(v) = u$;
- $\text{child}(u) = \{x, v\}$;
- $\text{leftChild}(u) = x$; $\text{rightChild}(u) = v$

T: $\text{subtree}(v)$



- $\text{height}(T) :=$ distance from root to the farthest leaf.
 - $\text{ht}(v) :=$ distance from v to the farthest leaf.
 - $\text{depth}(v) :=$ distance from the root to v .
- eg, $\text{ht}(T) = 3$, $\text{ht}(v) = \text{depth}(v) = 1$.
 $\text{ht}(q) = 0$, $\text{depth}(q) = 2$.
- ▷ $\forall$ nodes $v \in T$, $\text{ht}(v) + \text{depth}(v) \leq \text{ht}(T)$.
- $\text{ht}(\text{subtree}(x)) = 2$.
- ▷ $\forall$ nodes $v \in T$, $\text{ht}(\text{subtree}(v)) = \text{ht}(v)$.
- ▷ $\text{ht}(\text{root}) = \text{ht}(T)$ & $\text{depth}(\text{root}) = 0$.

▷ An array $A[0 \dots n-1]$ is a binary tree with $ht = n-1$.



- It's a skew binary tree; because at every node one child is missing.
⇒ Tree is too high, too thin!

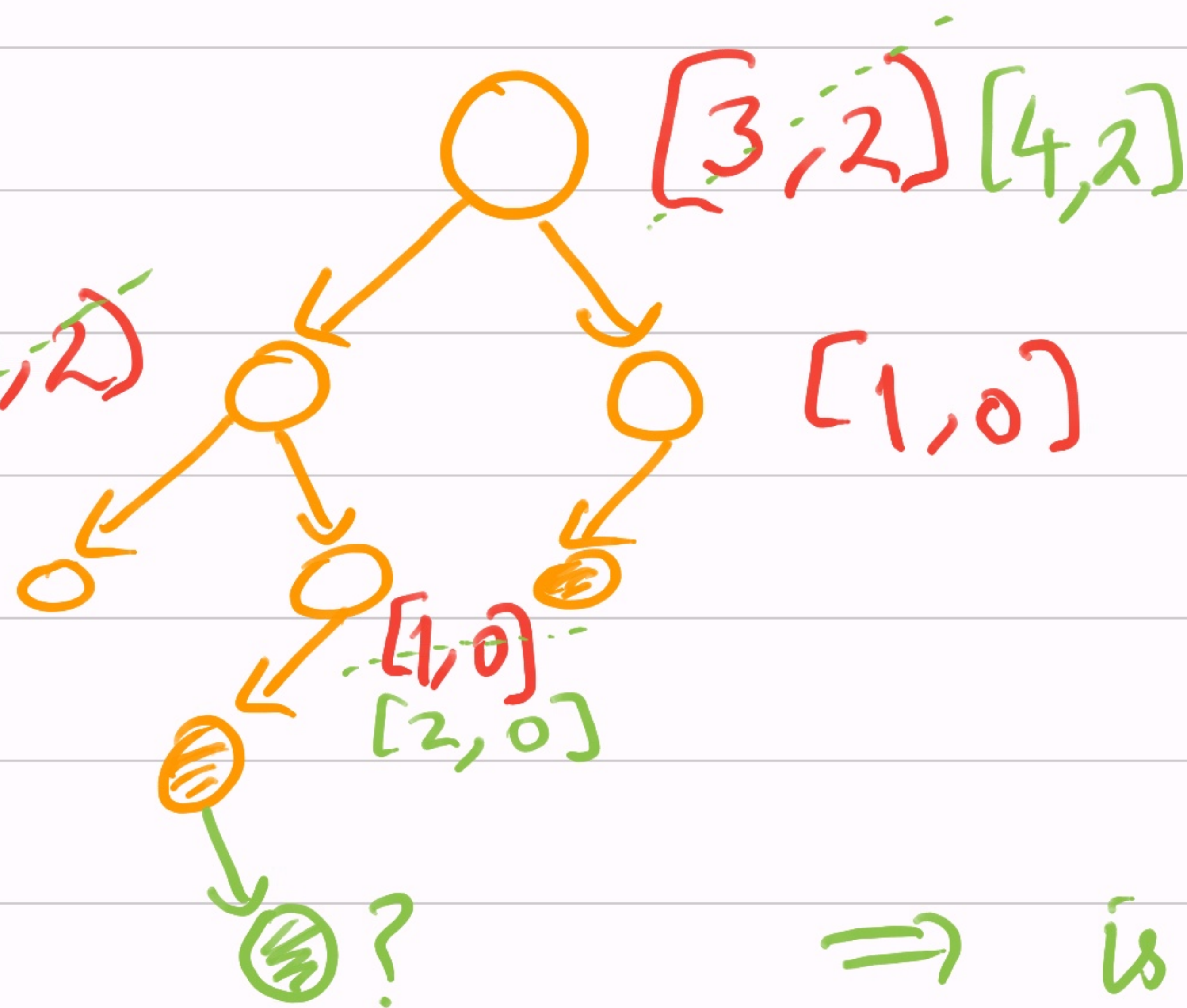
• Defn: A balanced binary tree has every node v s.t. $|ht(lsubtree(v)) - ht(rsubtree(v))| \leq 1$.

↳ or, consider the left/right-distances to the leaves:

- Ex.:

$[1,3]$

$[1,2]$



is a balanced tree.

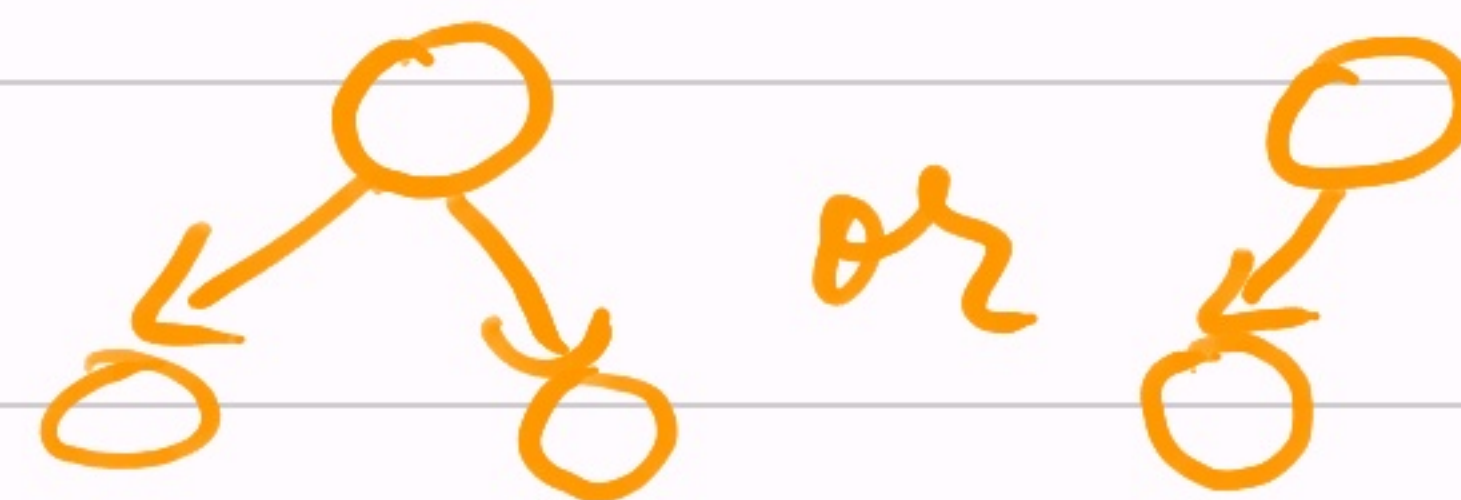
binary

$\Rightarrow$ is unbalanced tree

Height of a bbt (balanced binary tree)

• Let $N(h) :=$ min. #nodes in a bbt T of height $= h$.

• $N(1) = 2$ (base-case).



(bbl) • Consider $ht(T) = h+2$.

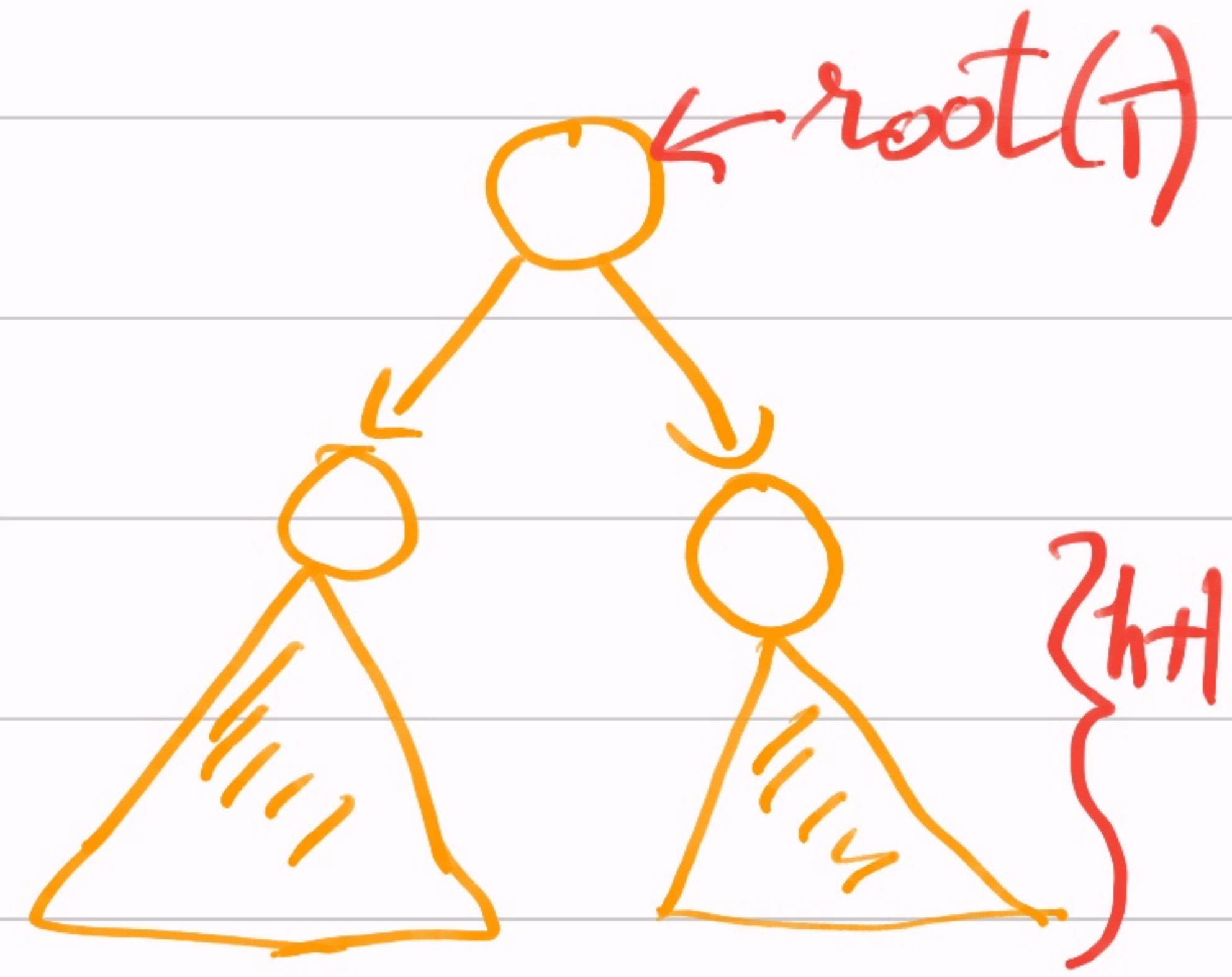
$$\Rightarrow N(h+2) \geq 1 + N(h+1) + N(h).$$

$\uparrow$
root

$\uparrow$
 $ht = h+1$
subtree

$\uparrow$
rest

$\left. \begin{array}{l} h \text{ or } h+1 \\ \text{(bbl)} \end{array} \right\}$



$$\Rightarrow N(h+2) > \text{Fib}(h+2) \geq 1.5^{h+1}.$$

$$\triangleright N(h) > 1.5^{h-1}.$$

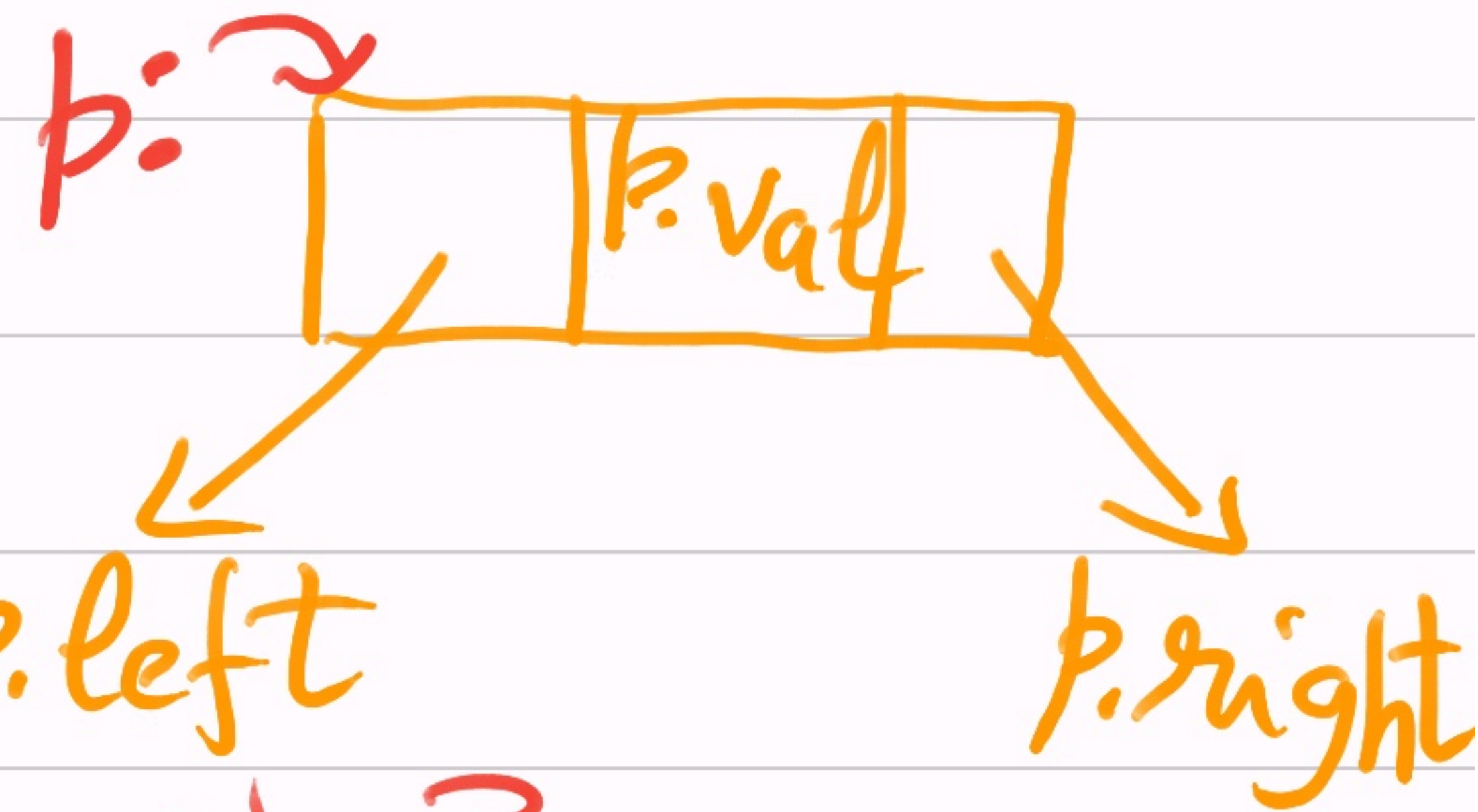
$$\Rightarrow h < O(\lg N(h))$$

Thm: For $|T|=n$, $h = O(\lg n)$. [space = $O(n)$]

Implementing a binary tree

- Keeping a pointer to the root (T).
- Each node (e.g., struct) keeps a val, ptr-left-child, & ptr-right-child.

↳ looks like doubly-link list implement.



Qn: So, how does it help in building the directory (search)?

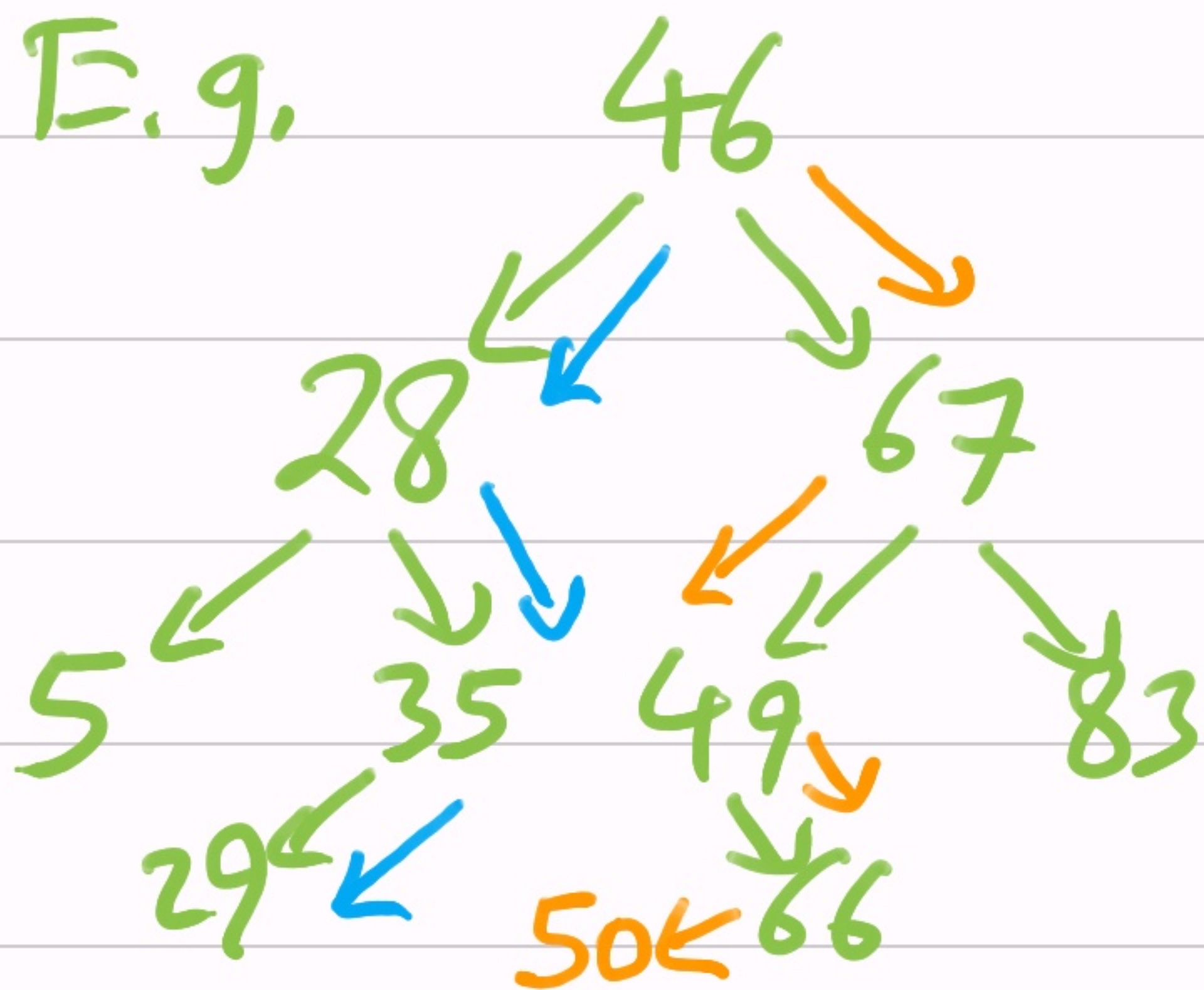
↳ One more concept is needed:

Binary Search Tree (bst)

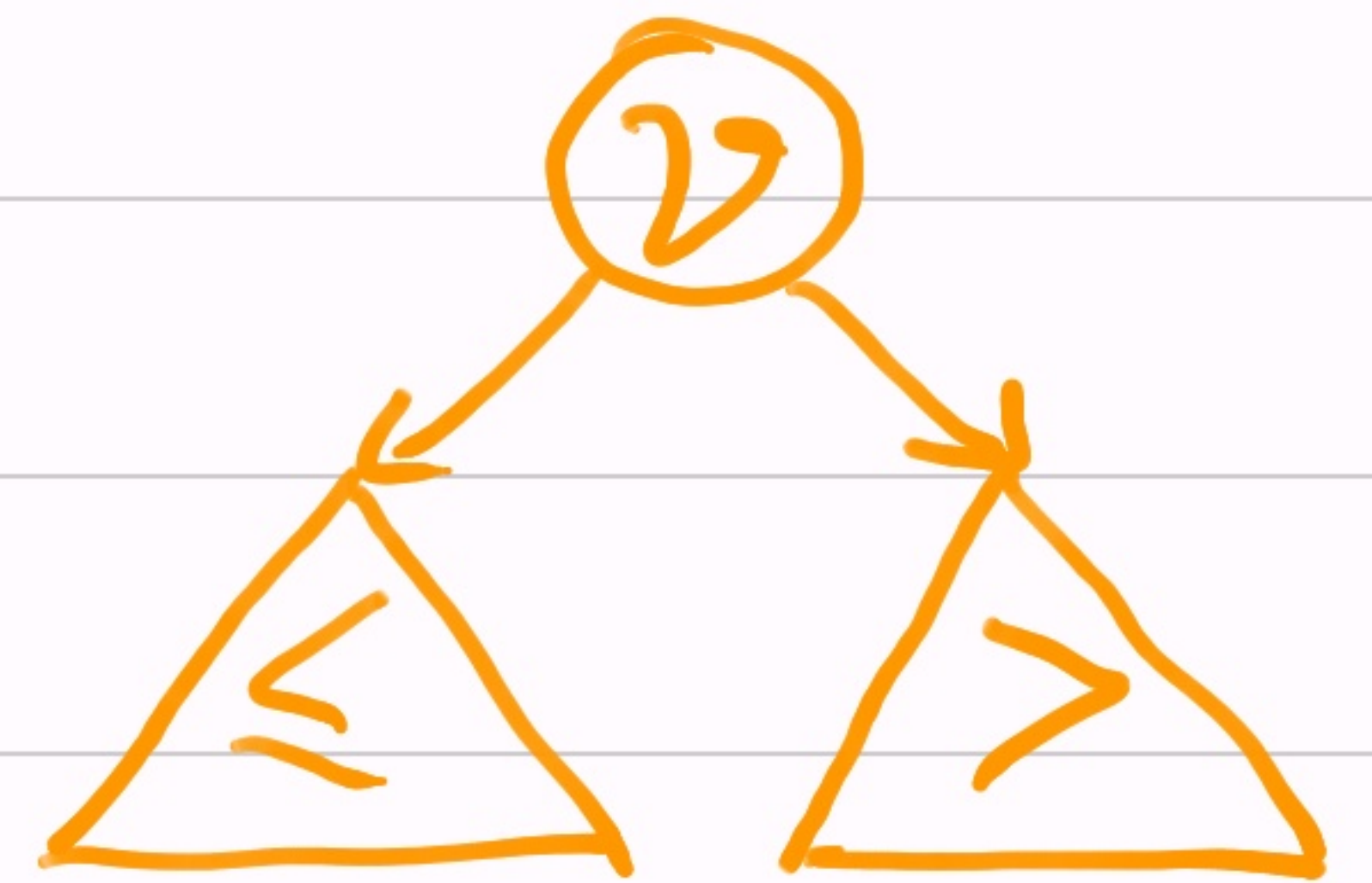
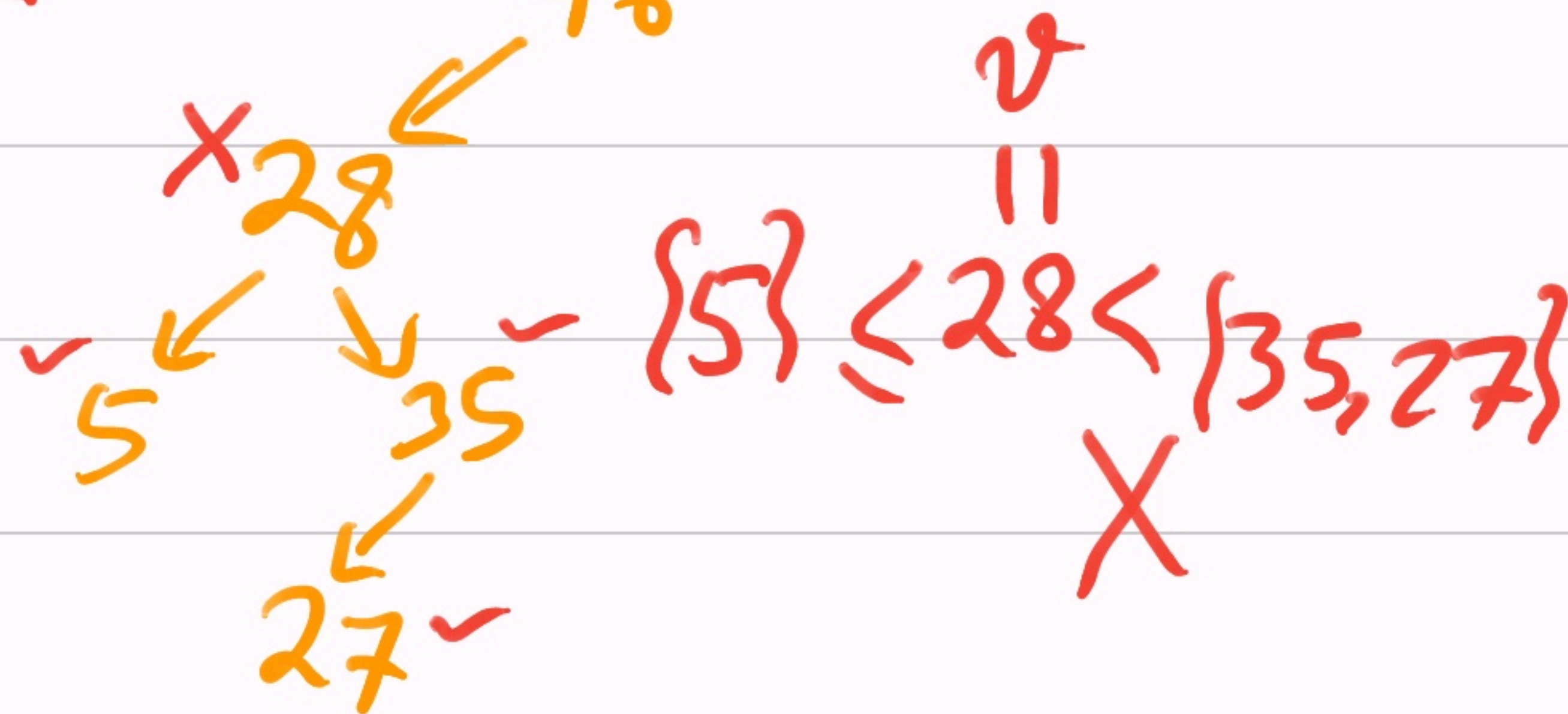
- Defn: A binary tree T is called bst if $\forall v \in T$:

- every value in $\text{subtree}(v, \text{left}) \leq v.\text{val}$,
- " " " " " $(v, \text{right}) >$ " .

E.g.



non-bst: 46 ✓



- E.g. search for 29: It's like doing binary-search in a "sorted array"! [It's found iff $x=29$ is present in bst.]

```
Search(T, x) { // T is bst & x is int.  
  p ← T; found ← FALSE;  
  while (!found & p ≠ NULL)  
    if (p.val = x) found ← TRUE;  
    else if (p.val < x) p ← p.right;  
    else p ← p.left;  
  return p; // p = location of x in T. }
```


▷ $\text{search}(T, x)$ will find x ; takes time $O(\text{depth}(p_x)) \leq O(\text{ht}(T))$.

- $\text{insert}(T, x)$, maintaining bst property:

- e.g. $\text{insert}(T, 50)$: do $\text{search}(T, 50)$ & then insert 50.

▷ $\text{insert}(T, x)$ takes time $O(\text{depth}(p_x)) \leq O(\text{ht}(T))$.

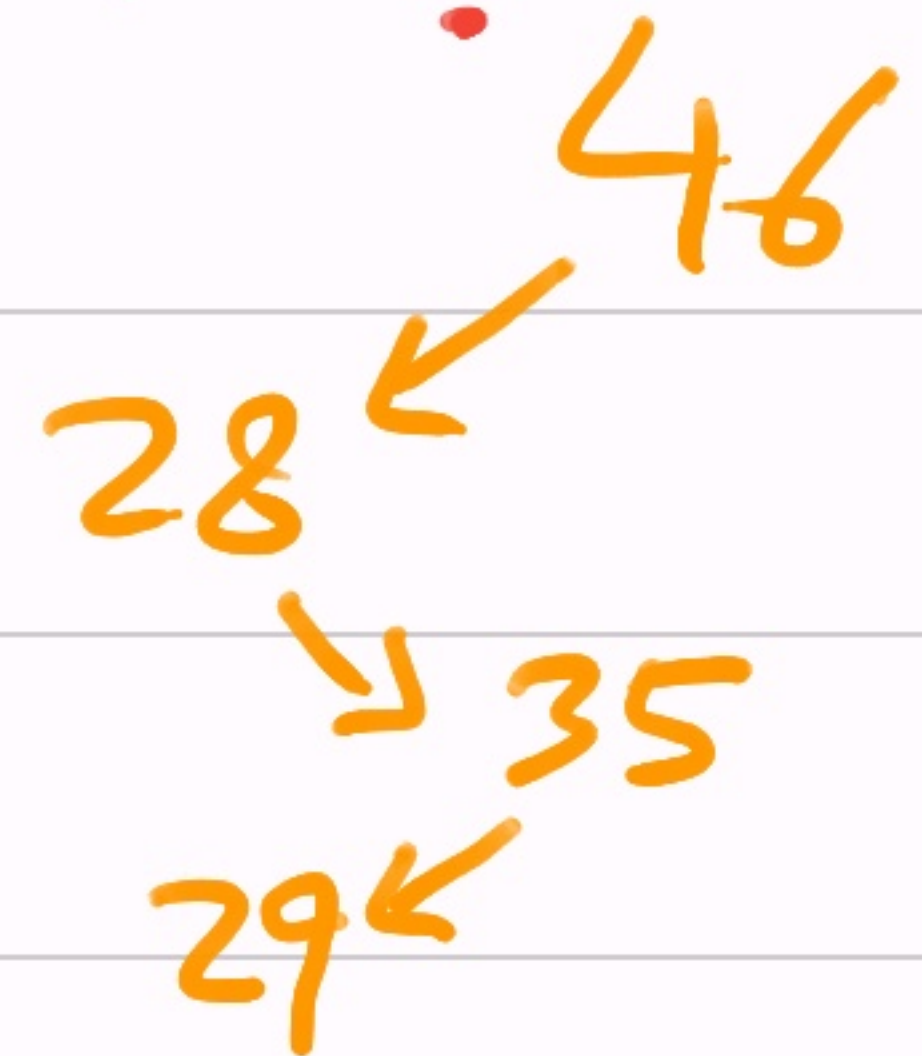
Exercise: (1) Pseudocode of $\text{insert}(T, x)$.
(2) Convert bst T into sorted array in $O(n)$ time.

(3) Convert a sorted array $A[0..n-1]$ into a balanced bst T in $O(n)$ time.

Hint (for (2)): Recursively print $\{\underbrace{v.\text{left}}_{\text{calls}}, \underbrace{v}_{\text{node}}, \underbrace{v.\text{right}}_{\text{calls}}\}$

- Qn: With this type of insert/search, can we say, $ht(T) = O(\lg n)$, $n = |T|$?

→ Insertion sequence can give $ht(T) = n-1$, in the worst-case.



→ Correction: insertion algo. needs to make sure that T is bbst. $\Rightarrow ht(T) = O(\lg|T|)$.

→ Red-Black-Tree: Idea of insertion is:
rotate to balance.



left-rotate
the unbalanced
subtree



- Before this, we do an easier ds - Stack.
- Problem: expression-evaluation eg.
 $x = 3 + 4 * (5 - 6 * (8 + 9))$. in 1-scan?!